STAT 639 - Classification Exercises

ISLR Chapter 4: Exercises 1, 4, 8, 14; Chapter 9: Exercise 3(a)-(f)

Bao Dang

September 07, 2026

Contents

1 Chapter 4 - Exercise 1 **2**

2 Chapter 4 - Exercise 4 **2**
2.1 (a) One feature . . . 2
2.2 (b) Two features . . . 2
2.3 (c) One hundred features . . . 3
2.4 (d) Interpretation . . . 3
2.5 (e) Hypercube containing 10% of observations . . . 3

3 Chapter 4 - Exercise 8 **3**

4 Chapter 4 - Exercise 14 **4**
4.1 (a) Create `mpg01` . . . 4
4.2 (b) Explore associations . . . 5
4.3 (c) Split into training and test sets . . . 6
4.4 (d) LDA . . . 7
4.5 (e) QDA . . . 7
4.6 (f) Logistic regression . . . 8
4.7 (g) Naive Bayes . . . 8
4.8 (h) KNN for several values of K . . . 9
4.8.1 Model comparison . . . 9

5 Chapter 9 - Exercise 3 **10**
5.1 (a) Sketch the observations . . . 10
5.2 (b) Optimal separating hyperplane . . . 11
5.3 (c) Classification rule . . . 12
5.4 (d) Margin . . . 13
5.5 (e) Support vectors . . . 14
5.6 (f) Moving observation 7 slightly . . . 14

1 Chapter 4 - Exercise 1

The logistic regression model can be written as

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1 + e^{\beta_0+\beta_1 X}}.$$

Let $\eta = \beta_0 + \beta_1 X$. Then

$$p(X) = \frac{e^{\eta}}{1 + e^{\eta}}.$$

Multiplying both sides by $1 + e^{\eta}$,

$$p(X)(1 + e^{\eta}) = e^{\eta}.$$

Therefore,

$$p(X) = e^{\eta}(1 - p(X)),$$

so

$$\frac{p(X)}{1 - p(X)} = e^{\eta} = e^{\beta_0+\beta_1 X}.$$

Taking the natural logarithm gives

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X.$$

Thus the logistic-function representation and the logit representation are equivalent.

2 Chapter 4 - Exercise 4

This exercise illustrates the **curse of dimensionality** for local methods such as KNN.

2.1 (a) One feature

The neighborhood covers 10% of the range of X. Since X is uniformly distributed on $[0, 1]$, the expected fraction of observations in the neighborhood is

$$0.10 = 10\%.$$

2.2 (b) Two features

An observation must fall within 10% of the range for both X_1 and X_2. Therefore the expected fraction is

$$0.10 \times 0.10 = 0.01 = 1\%.$$

2.3 (c) One hundred features

For $p = 100$, the expected fraction is

$$(0.10)^{100} = 10^{-100}.$$

This is essentially zero.

```
0.1^100
```

```
## [1] 1e-100
```

2.4 (d) Interpretation

As the number of predictors increases, the fraction of observations that are simultaneously close to a test observation in every dimension decreases extremely quickly. Therefore, in high dimensions, KNN has very few genuinely nearby training observations available for prediction. To obtain enough neighbors, the neighborhood must become much larger, so the observations are no longer truly local.

2.5 (e) Hypercube containing 10% of observations

If each side of a p-dimensional hypercube has length s, its volume is s^p. We want

$$s^p = 0.10,$$

so

$$s = 0.10^{1/p}.$$

```
p <- c(1, 2, 100)
side_length <- 0.1^(1 / p)
data.frame(p = p, side_length = side_length)
```

```
##     p side_length
## 1   1   0.1000000
## 2   2   0.3162278
## 3 100   0.9772372
```

Hence:

- $p = 1$: $s = 0.10$
- $p = 2$: $s = \sqrt{0.10} \approx 0.316$
- $p = 100$: $s = 0.10^{1/100} \approx 0.977$

For 100 predictors, the neighborhood must span about 97.7% of the range of **each** predictor just to contain 10% of the observations on average. This shows why the idea of a "local" neighborhood breaks down in high dimensions.

3 Chapter 4 - Exercise 8

The logistic regression test error is given directly as **30%**.

For 1-nearest neighbors, the training error is 0% because each training observation is its own nearest neighbor. Let E_{test} be the KNN test error. Because the training and test sets have equal size and the average error across both sets is 18%,

$$\frac{0 + E_{test}}{2} = 0.18.$$

Therefore,

$$E_{test} = 0.36.$$

So the 1-NN test error is **36%**, compared with **30%** for logistic regression. We should prefer **logistic regression** for classifying new observations because test-set performance is the relevant measure of generalization.

4 Chapter 4 - Exercise 14

This exercise uses the `Auto` data from the `ISLR2` package.

```
if (!requireNamespace("ISLR2", quietly = TRUE)) {
  install.packages("ISLR2")
}
if (!requireNamespace("MASS", quietly = TRUE)) {
  install.packages("MASS")
}
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071")
}
if (!requireNamespace("class", quietly = TRUE)) {
  install.packages("class")
}

library(ISLR2)
library(MASS)
library(e1071)
library(class)

data("Auto")
```

4.1 (a) Create `mpg01`

```
mpg_median <- median(Auto$mpg)
mpg01 <- ifelse(Auto$mpg > mpg_median, 1, 0)

auto <- data.frame(mpg01 = factor(mpg01), Auto)

table(auto$mpg01)
```

```
##
##   0   1
## 196 196
```

```
mpg_median
```

```
## [1] 22.75
```

`mpg01 = 1` represents cars with mileage above the sample median, while `mpg01 = 0` represents cars at or below the median.

4.2 (b) Explore associations

```
par(mfrow = c(2, 3))

plot(auto$mpg01, auto$cylinders,
     xlab = "mpg01", ylab = "Cylinders",
     main = "Cylinders vs mpg01")

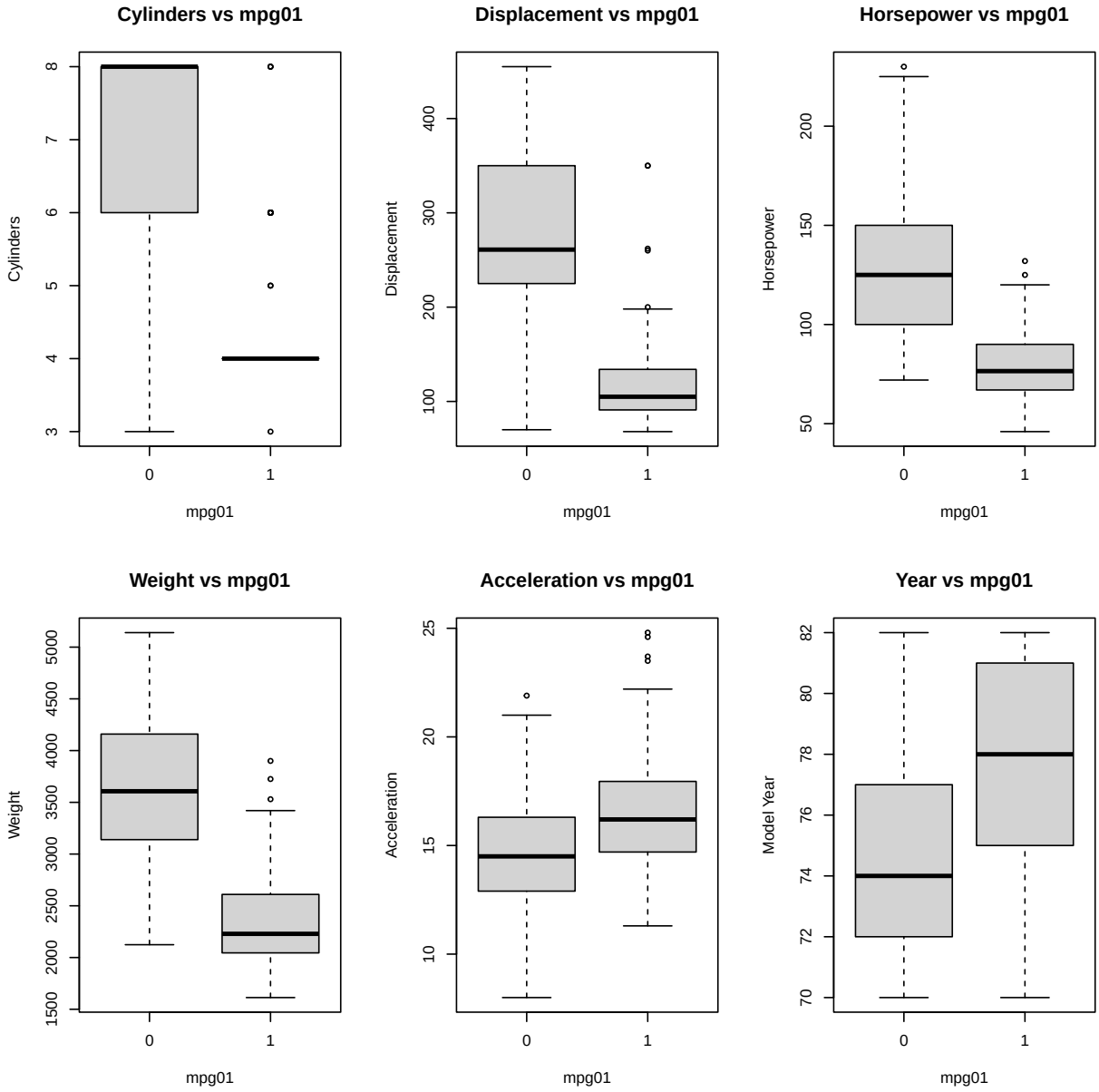
plot(auto$mpg01, auto$displacement,
     xlab = "mpg01", ylab = "Displacement",
     main = "Displacement vs mpg01")

plot(auto$mpg01, auto$horsepower,
     xlab = "mpg01", ylab = "Horsepower",
     main = "Horsepower vs mpg01")

plot(auto$mpg01, auto$weight,
     xlab = "mpg01", ylab = "Weight",
     main = "Weight vs mpg01")

plot(auto$mpg01, auto$acceleration,
     xlab = "mpg01", ylab = "Acceleration",
     main = "Acceleration vs mpg01")

plot(auto$mpg01, auto$year,
     xlab = "mpg01", ylab = "Model Year",
     main = "Year vs mpg01")
```




```
par(mfrow = c(1, 1))
```

The clearest differences are typically seen for `cylinders`, `displacement`, `horsepower`, and `weight`: high-mileage cars tend to have fewer cylinders, smaller displacement, lower horsepower, and lower weight. `year` also shows an association, with newer cars tending to have higher mileage. I will use `cylinders`, `displacement`, `horsepower`, and `weight` as predictors below.

4.3 (c) Split into training and test sets

A reproducible 70/30 random split is used.

```
set.seed(639)

train_id <- sample(
  seq_len(nrow(auto)),
```

```
  size = floor(0.70 * nrow(auto))
)

test_id <- setdiff(seq_len(nrow(auto)), train_id)

train <- auto[train_id, ]
test <- auto[test_id, ]

c(training = nrow(train), test = nrow(test))
```

```
## training     test
##      274      118
```

4.4 (d) LDA

```
lda_fit <- lda(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train
)

lda_pred <- predict(lda_fit, newdata = test)$class
lda_cm <- table(Predicted = lda_pred, Actual = test$mpg01)
lda_error <- mean(lda_pred != test$mpg01)

lda_cm
```

```
##          Actual
## Predicted  0  1
##         0 53  1
##         1 11 53
```

```
lda_error
```

```
## [1] 0.1016949
```

The LDA test error is 0.1017 (about 10.2%).

4.5 (e) QDA

```
qda_fit <- qda(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train
)

qda_pred <- predict(qda_fit, newdata = test)$class
qda_cm <- table(Predicted = qda_pred, Actual = test$mpg01)
qda_error <- mean(qda_pred != test$mpg01)

qda_cm
```

```
##          Actual
## Predicted  0  1
##         0 55  2
##         1  9 52
```

```
qda_error
```

```
## [1] 0.09322034
```

The QDA test error is 0.0932 (about 9.3%).

4.6 (f) Logistic regression

```
logit_fit <- glm(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train,
  family = binomial
)

logit_prob <- predict(logit_fit, newdata = test, type = "response")
logit_pred <- factor(ifelse(logit_prob > 0.5, 1, 0), levels = levels(test$mpg01))

logit_cm <- table(Predicted = logit_pred, Actual = test$mpg01)
logit_error <- mean(logit_pred != test$mpg01)

logit_cm
```

```
##          Actual
## Predicted  0  1
##         0 53  1
##         1 11 53
```

```
logit_error
```

```
## [1] 0.1016949
```

The logistic-regression test error is 0.1017 (about 10.2%).

4.7 (g) Naive Bayes

```
nb_fit <- naiveBayes(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train
)

nb_pred <- predict(nb_fit, newdata = test)
nb_cm <- table(Predicted = nb_pred, Actual = test$mpg01)
nb_error <- mean(nb_pred != test$mpg01)

nb_cm
```

```
##          Actual
## Predicted  0  1
##         0 53  1
##         1 11 53
```

```
nb_error
```

```
## [1] 0.1016949
```

The naive-Bayes test error is 0.1017 (about 10.2%).

4.8 (h) KNN for several values of K

KNN is distance-based, so the predictors are standardized using the **training-set** means and standard deviations. The same transformation is then applied to the test set.

```
vars <- c("cylinders", "displacement", "horsepower", "weight")

x_train <- train[, vars]
x_test <- test[, vars]

train_mean <- sapply(x_train, mean)
train_sd <- sapply(x_train, sd)

x_train_scaled <- scale(x_train, center = train_mean, scale = train_sd)
x_test_scaled <- scale(x_test, center = train_mean, scale = train_sd)

k_values <- c(1, 3, 5, 7, 9, 15, 25)

knn_errors <- sapply(k_values, function(k) {
  pred <- knn(
    train = x_train_scaled,
    test = x_test_scaled,
    cl = train$mpg01,
    k = k
  )
  mean(pred != test$mpg01)
})

knn_results <- data.frame(
  K = k_values,
  Test_Error = knn_errors
)

knn_results
```

```
##    K Test_Error
## 1  1 0.05084746
## 2  3 0.08474576
## 3  5 0.08474576
## 4  7 0.09322034
## 5  9 0.09322034
## 6 15 0.09322034
## 7 25 0.09322034
```

```
best_knn <- knn_results[which.min(knn_results$Test_Error), ]
best_knn
```

```
##   K Test_Error
## 1 1 0.05084746
```

For this split, the best tested value is $K = 1$, with test error 0.0508 (about 5.1%).

4.8.1 Model comparison

```
model_results <- data.frame(
  Method = c("LDA", "QDA", "Logistic Regression", "Naive Bayes", paste0("KNN (K=", best_knn$K, ")")),
```

```
  Test_Error = c(lda_error, qda_error, logit_error, nb_error, best_knn$Test_Error)
)

model_results[order(model_results$Test_Error), ]
```

```
##                Method Test_Error
## 5           KNN (K=1) 0.05084746
## 2                 QDA 0.09322034
## 1                 LDA 0.10169492
## 3 Logistic Regression 0.10169492
## 4         Naive Bayes 0.10169492
```

The methods all use the same training/test split, so their test errors can be compared directly. The method at the top of the table has the lowest test error for this particular split.

5 Chapter 9 - Exercise 3

We have seven observations in two dimensions.

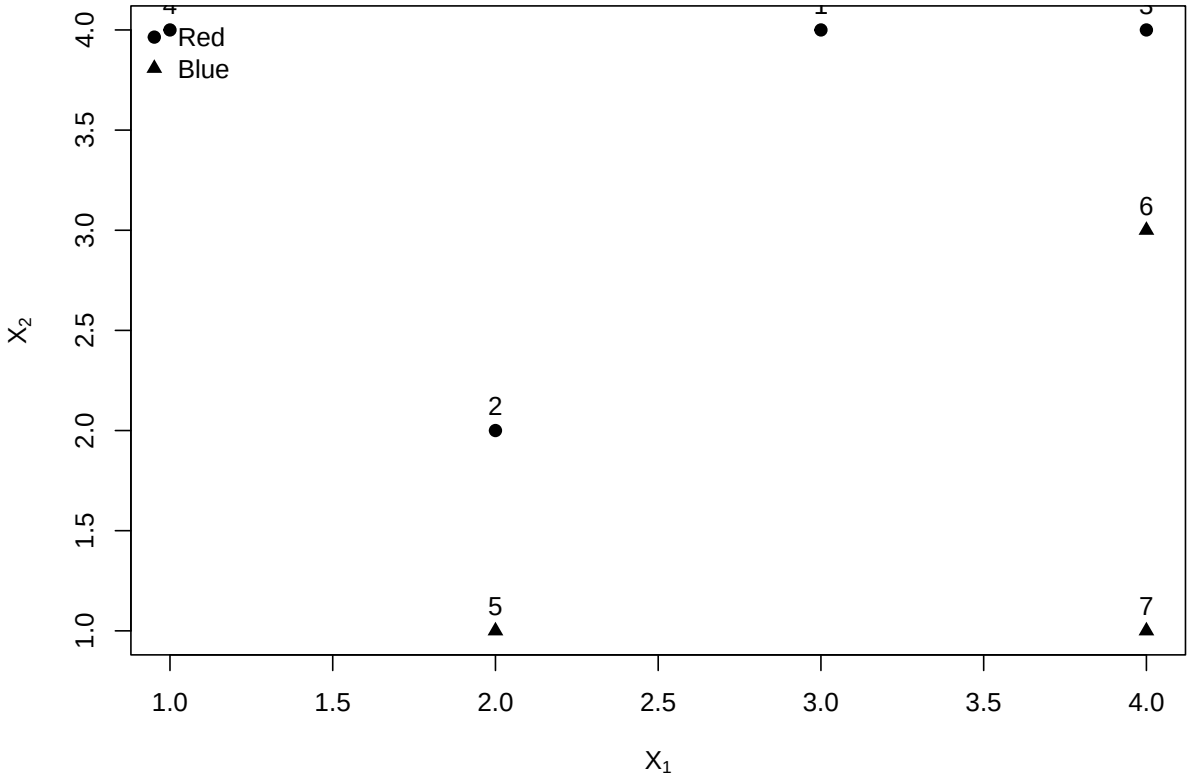
```
svm_data <- data.frame(
  X1 = c(3, 2, 4, 1, 2, 4, 4),
  X2 = c(4, 2, 4, 4, 1, 3, 1),
  Class = factor(c("Red", "Red", "Red", "Red", "Blue", "Blue", "Blue")),
  Obs = 1:7
)

svm_data
```

```
##   X1 X2 Class Obs
## 1  3  4   Red   1
## 2  2  2   Red   2
## 3  4  4   Red   3
## 4  1  4   Red   4
## 5  2  1  Blue   5
## 6  4  3  Blue   6
## 7  4  1  Blue   7
```

5.1 (a) Sketch the observations

```
plot(
  svm_data$X1,
  svm_data$X2,
  pch = ifelse(svm_data$Class == "Red", 19, 17),
  xlab = expression(X[1]),
  ylab = expression(X[2]),
  main = "Chapter 9, Exercise 3"
)
text(svm_data$X1, svm_data$X2, labels = svm_data$Obs, pos = 3)
legend("topleft", legend = c("Red", "Blue"), pch = c(19, 17), bty = "n")
```



5.2 (b) Optimal separating hyperplane

A convenient form of the maximal-margin hyperplane is

$$X_1 - X_2 = 0.5,$$

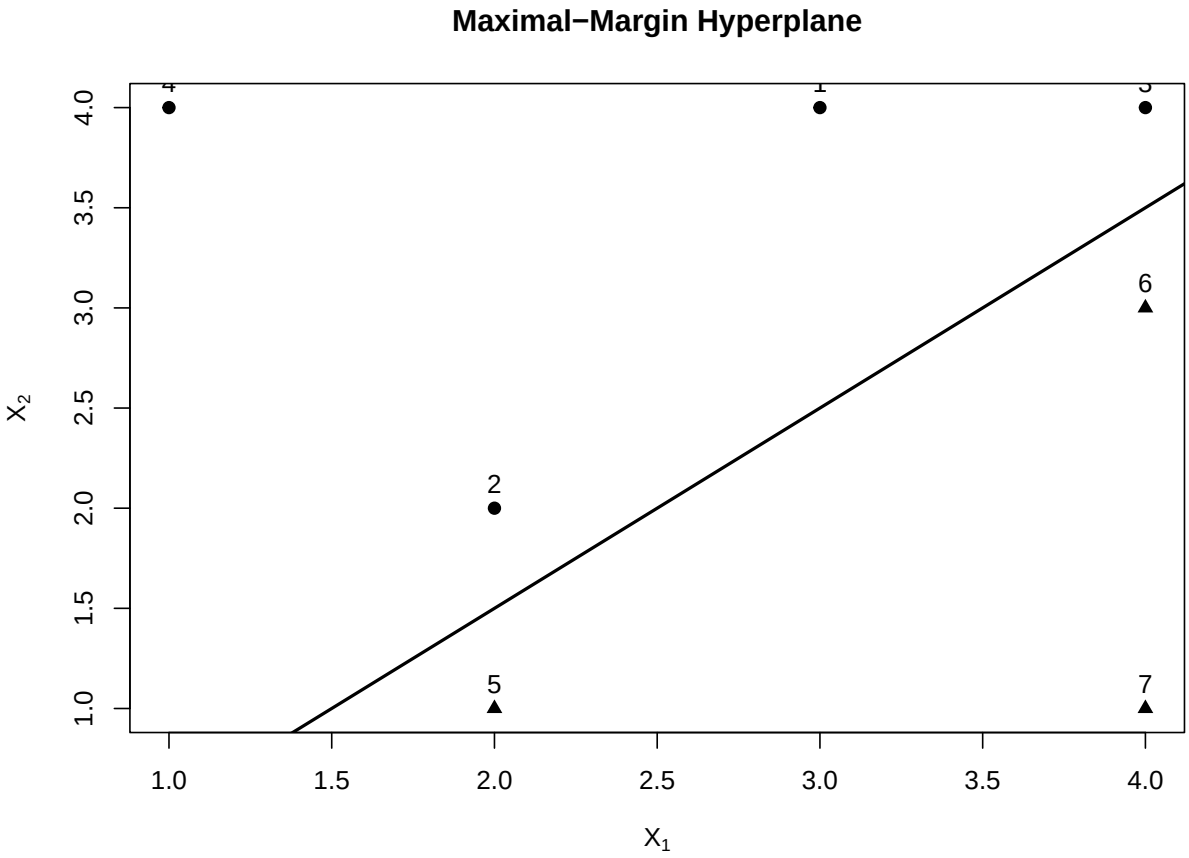
or equivalently

$$0.5 - X_1 + X_2 = 0.$$

The nearest red observations satisfy $X_1 - X_2 = 0$, while the nearest blue observations satisfy $X_1 - X_2 = 1$. The maximal-margin boundary lies halfway between these two parallel lines.

```
plot(
  svm_data$X1,
  svm_data$X2,
  pch = ifelse(svm_data$Class == "Red", 19, 17),
  xlab = expression(X[1]),
  ylab = expression(X[2]),
  main = "Maximal-Margin Hyperplane"
)
text(svm_data$X1, svm_data$X2, labels = svm_data$Obs, pos = 3)
```

```r
# X1 - X2 = 0.5  ->  X2 = X1 - 0.5
abline(a = -0.5, b = 1, lwd = 2)
```



5.3 (c) Classification rule

Using

$$\beta_0 = 0.5, \qquad \beta_1 = -1, \qquad \beta_2 = 1,$$

the classifier is

$$0.5 - X_1 + X_2 = 0.$$

Therefore:

- classify an observation as **Red** if $0.5 - X_1 + X_2 > 0$;
- classify it as **Blue** otherwise.

```r
score <- 0.5 - svm_data$X1 + svm_data$X2
predicted <- ifelse(score > 0, "Red", "Blue")
data.frame(svm_data, score = score, predicted = predicted)
```

```
##   X1 X2 Class Obs score predicted
## 1  3  4   Red   1   1.5       Red
```

```
## 2  2  2   Red   2   0.5       Red
## 3  4  4   Red   3   0.5       Red
## 4  1  4   Red   4   3.5       Red
## 5  2  1  Blue   5  -0.5      Blue
## 6  4  3  Blue   6  -0.5      Blue
## 7  4  1  Blue   7  -2.5      Blue
```

5.4 (d) Margin

The two margin boundaries are

$$X_1 - X_2 = 0$$

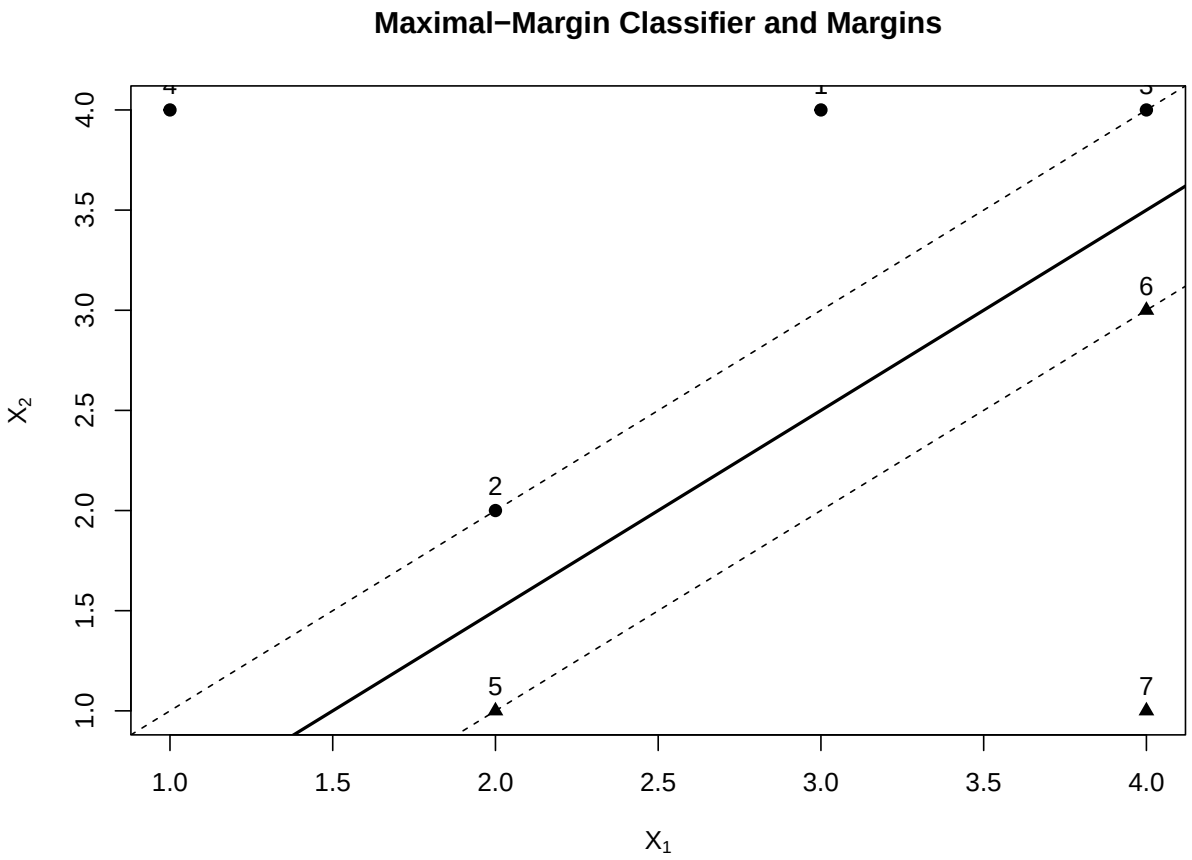
and

$$X_1 - X_2 = 1.$$

The separating hyperplane $X_1 - X_2 = 0.5$ lies exactly halfway between them.

```r
plot(
  svm_data$X1,
  svm_data$X2,
  pch = ifelse(svm_data$Class == "Red", 19, 17),
  xlab = expression(X[1]),
  ylab = expression(X[2]),
  main = "Maximal-Margin Classifier and Margins"
)
text(svm_data$X1, svm_data$X2, labels = svm_data$Obs, pos = 3)

abline(a = -0.5, b = 1, lwd = 2)  # decision boundary
abline(a = 0, b = 1, lty = 2)     # red margin: X2 = X1
abline(a = -1, b = 1, lty = 2)    # blue margin: X2 = X1 - 1
```



5.5 (e) Support vectors

The support vectors are the observations lying on the two margin boundaries:

- Red: observations **2** and **3**, where $X_1 - X_2 = 0$
- Blue: observations **5** and **6**, where $X_1 - X_2 = 1$

```
svm_data[c(2, 3, 5, 6), ]
```

```
##   X1 X2 Class Obs
## 2  2  2   Red   2
## 3  4  4   Red   3
## 5  2  1  Blue   5
## 6  4  3  Blue   6
```

5.6 (f) Moving observation 7 slightly

Observation 7 is $(4, 1)$, so

$$X_1 - X_2 = 3.$$

It lies well inside the Blue side and is not a support vector. The maximal-margin hyperplane is determined by the observations closest to the boundary, namely observations 2, 3, 5, and 6. Therefore, a **small movement** of observation 7 that leaves it away from the margin will not change the maximal-margin hyperplane.